

## Lista de Exercícios: Especializando Comportamentos com Sobrescrita

### 1. Sistema de Pedágio "Via Rápida"

A concessionária "Via Rápida" precisa de um sistema para calcular as tarifas de pedágio, que variam conforme o tipo de veículo.

- **Sua Tarefa:** Implemente a hierarquia de classes Veiculo, Carro e Moto, conforme o desafio da aula.
  - **Veiculo (Superclasse):** Deve ter atributos como marca e modelo. Crie um método public double calcularPedagio() que retorna uma tarifa base de R\$ 10,50.
  - **Carro (Subclasse):** Deve herdar de Veiculo. Sobrescreva (@Override) o método calcularPedagio() para que ele retorne a tarifa específica de R\$ 12,00.
  - **Moto (Subclasse):** Deve herdar de Veiculo. Sobrescreva (@Override) o método calcularPedagio() para que ele retorne a tarifa de R\$ 6,00.
  - **Teste:** No main, crie um objeto de cada tipo e chame o mesmo método calcularPedagio() em cada um para verificar se os valores corretos são retornados.

### 2. Simulador de Sons de Animais

Você está desenvolvendo um software educativo para crianças que simula os sons de diferentes animais.

- **Sua Tarefa:** Crie uma superclasse Animal com um método emitirSom() que imprime "O animal emite um som". Em seguida, crie duas subclasses:
  - **Cachorro:** Sobrescreva o método emitirSom() para que ele imprima "Au au!".
  - **Gato:** Sobrescreva o método emitirSom() para que ele imprima "Miau!".
  - **Teste:** No main, crie um Cachorro e um Gato e chame o método emitirSom() em ambos para observar os comportamentos diferentes.

### 3. Software de Design Gráfico "CanvasPro"

O "CanvasPro" precisa calcular a área de diferentes formas geométricas. A fórmula varia para cada forma, mas a ação de "calcular a área" é comum a todas.

- **Sua Tarefa:** Crie uma superclasse FormaGeometrica. Dentro dela, crie um método calcularArea() que retorna 0.0. Crie duas subclasses:
  - **Circulo:** Adicione o atributo raio e sobrescreva calcularArea() para retornar a fórmula correta ( $\text{Math.PI} * \text{raio} * \text{raio}$ ).
  - **Quadrado:** Adicione o atributo lado e sobrescreva calcularArea() para retornar a fórmula correta ( $\text{lado} * \text{lado}$ ).
  - **Teste:** No main, crie um círculo e um quadrado com medidas diferentes e imprima o resultado de calcularArea() para cada um.

### 4. Sistema de RH: Política de Férias

Uma empresa de tecnologia tem uma política de férias base, mas cargos diferentes têm benefícios adicionais.

- **Sua Tarefa:** Crie uma superclasse Colaborador com um método calcularFerias() que retorna o valor 30 (dias). Crie uma subclasse Gerente que herda de Colaborador. Sobrescreva o método calcularFerias() na classe Gerente para que ele retorne  $30 + 7$ , representando 30 dias padrão mais 7 dias de bônus.

### 5. Jogo de RPG "Crônicas de Valéria"

No jogo "Crônicas de Valéria", todos os personagens podem atacar, mas cada classe de personagem tem uma forma de ataque diferente.

- **Sua Tarefa:** Crie uma superclasse `Personagem` com um método `atacar()` que imprime "Personagem realiza um ataque básico!". Crie duas subclasses:
  - **Guerreiro:** Sobrescreva o método `atacar()` para imprimir "Guerreiro ataca com a espada!".
  - **Mago:** Sobrescreva o método `atacar()` para imprimir "Mago lança uma bola de fogo!".
  - **Teste:** Crie um `Guerreiro` e um `Mago` e chame o método `atacar()` em cada um.

## 6. Orquestra Virtual

Você está criando um software que simula uma orquestra. Cada instrumento pode ser "tocado", mas produz um som diferente.

- **Sua Tarefa:** Crie uma superclasse `InstrumentoMusical` com um método `tocar()` que imprime "Tocando um instrumento...". Crie subclasses como `Violao` (que imprime "Soando cordas de nylon..."), `Bateria` (que imprime "Rufo de tambores...") e `Piano` (que imprime "Melodia ao piano..."), todas sobrescrevendo o método `tocar()`.

## 7. Previsão de Código (Leitura e Análise)

Analise a hierarquia de classes abaixo. Sem compilar, preveja exatamente o que será impresso no console quando o método `main` for executado. Justifique sua resposta em um comentário.

```

class A {
    public void mostrarMensagem() {
        System.out.println("Mensagem da classe A");
    }
}

class B extends A {
    @Override
    public void mostrarMensagem() {
        System.out.println("Mensagem da classe B");
    }
}

class C extends B {
    // Esta classe NÃO sobreescreve o método.
}

// No main:
// public static void main(String[] args) {
//     A objA = new A();
//     B objB = new B();
//     C objC = new C();
//
//     objA.mostrarMensagem();
//     objB.mostrarMensagem();
//     objC.mostrarMensagem(); // Qual método será chamado aqui?
// }

```

## 8. O Anti-Padrão (Análise de Design)

Seu colega de equipe implementou a lógica de cálculo de impostos de um e-commerce da seguinte forma:

```

// Na classe Produto:
public double calcularImpostoProdutoComum() { return this.preco * 0.10; }

// Na classe Eletronico extends Produto:
public double calcularImpostoDeEletronico() { return this.preco * 0.15; }

// Na classe Alimento extends Produto:
public double calcularImpostoDeAlimento() { return this.preco * 0.05; }

```

- **Sua Tarefa:** Em um comentário no seu código, explique por que essa abordagem, com nomes de métodos diferentes, é um mau design. Descreva o principal problema que ocorrerá quando alguém precisar calcular o imposto total de um carrinho de compras com produtos de diferentes tipos. Qual seria a solução correta usando os conceitos desta aula?

## 9. Sobrecarga vs. Sobrescrita (Conceitual)

Pesquise rapidamente e explique com suas palavras a diferença fundamental entre Sobrecarga de Método (Overloading), que vimos em aulas passadas (ex: múltiplos construtores), e Sobrescrita de Método (Overriding), que vimos hoje. Dê um exemplo de código para cada um.

## 10. Desafio: Bônus com Reutilização de Lógica

No nosso sistema de RH, a regra do bônus para Gerente mudou. Agora é: "o bônus padrão de Funcionario (5%) MAIS um bônus de liderança fixo de R\$ 1000,00".

- **Sua Tarefa:** Sobrescreva o método `getBonus()` na classe `Gerente`. Dentro dele, em vez de recalcular a porcentagem, você deve **chamar o método `getBonus()` da superclasse** para obter o valor base e, em seguida, somar o valor adicional. (Dica: use a palavra-chave `super.getBonus()`). Esta é uma técnica muito comum para estender um comportamento sem duplicar a lógica da superclasse.